Arya Jain

Healthcare_Analytics_Report.pdf

Quick Submit

Quick Submit

Symbiosis International University

Document Details

Submission ID
trn:oid:::1:3689204301

Submission Date
Oct 24, 2026, 05:24 PM GMT+5:30

Download Date
Oct 24, 2026, 05:45 PM GMT+5:30

File Name
Healthcare_Analytics_Report.pdf

File Size
389.7 KB

7 Pages

5,640 Words

27,884 Characters

12% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

12 AI-generated 12%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

prediction of early readmissions; and (4) what features are most important in our learned 'hospital tree' model? This study is an exploration of the way different models behave, but we're not suggesting a new way to make clinical decisions.

II. DATASETS AND TARGETS

Dataset 1 - Diabetes Health Indicators (BRFSS derived): The CSV file had 253,680 rows of data and 22 columns.

The target - Diabetes_012 - had 3 classes: people with diabetes, people with pre-diabetes and people with no sign of diabetes. Our analysis filtered out 23,899 duplicates and ended up with 229,781 rows and 21 predictors to work with from the original 22.

Dataset 2 - Diabetes 130-US Hospitals: We had another CSV file to deal with - a whopping 101,766 hospital encounters and 48 columns to sift through. The readmitted field originally had a pretty messy set of labels - NO, >30, and <30. We tidied that up by turning it into a simple binary target variable: if a patient got readmitted within 30 days and got a 1, everyone else got a 0. We also pruned a couple of columns from the list of predictors that weren't doing much for us - encounter_id and patient_nbr. And as for the 374,017 missing values - out came the '?' symbols and in went NaNs.

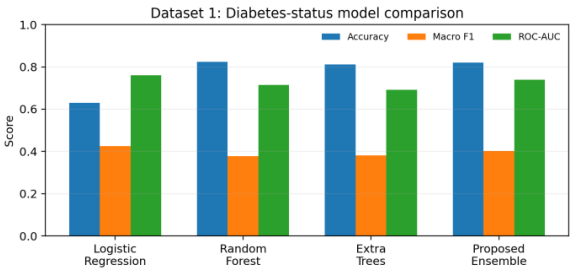


Fig. 1. Model comparison for Dataset 1 - the scores are based on the notebook's saved test-set comparison.

Table I. Dataset description, target definition and split sizes.

Dataset	Original records	Columns	Cleaning / target	Model features	Train / test
Diabetes Health Indicators	253,680	22	Drop 23,899 duplicates; 3-class Diabetes_012	21	183,824 / 45,957
Diabetes 130-US Hospitals	101,766	48	Map '?' to missing; <30 = 1, other labels = 0	47 (IDs excluded)	81,412 / 20,354

Dataset 1: Diabetes Health Indicators: The CSV data for this one had a huge number of records - 253,680, and 22 columns to boot. Once we'd tidied up the data a bit, it boiled down to three clear categories for outcomes: 0 - no diabetes, 1 - prediabetes, and 2 - diabetes. But before we could do that, we had to chop off 23,899 duplicate rows - once we'd done that we were left with 229,781 records. That trimmed down the extra rows but didn't exactly address the issues that come with working with survey based data. And as anyone who's worked with this kind of data will tell you, one thing it's not short on is limitations. The classes in this dataset are also pretty heavily skewed - 213,703 records in class 0, 4,631 in class 1 and a whopping 35,346 in class 2. With prediabetes making up such a small minority, we really need to be careful and use measures that take that into account.

Dataset 2: Diabetes 130-US Hospitals: This hospital file is looking at 101,766 encounters and has a massive 48 columns to navigate. The readmitted variable has labels that are a bit of a mess - NO, >30 and 30. But for the purposes of this project, we're looking at it in a much simpler way: if the label is 30, we say yes, if it's NO or >30, we say no. So we're not looking at whether the patient got readmitted a month down the line, but whether they got readmitted within the 30 day window. Of the 54,864 records with the original labels, 35,545 were in the >30 category and 11,357 were in the 30 category - which is roughly 11.2% of the 'no' category, if you can even call it that .All the missing values in the form of ? in the hospital dataset were treated as missing - and it looks like there's 374,017 of them in the supplied file. To avoid the model learning record specific stuff we excluded fields like encounter_id and patient_nbr from the predictors. This analysis is looking at the individual encounter records, not the patients overall. And with that we may end up splitting some patients records between two groups, a problem that should be addressed in a patient grouped validation down the line.

III. PREPROCESSING AND MODELS

Both datasets were split up using Train/Test Split with a test size of 0.20 and a random seed of 42 to ensure things are kept consistent. The feature imputation in the pre-processing pipeline is handled by scikit-learn. The numerical features were treated with the median of the training data, then standardised. Categorisation features on the other hand used the most frequent category from the training set and were one-hot encoded - any unknowns were simply ignored. This pipeline helps prevent leakage by allowing the model to learn its parameters as it goes.

Four different classifiers were compared for each task: number one was a Logistic Regression with a high max iteration count of 1000 and class weights balanced to steer it away from bias; number two was a Random Forest with 100 trees, balanced class weighting and parallel execution; number 3 was an Extra Trees model with a 100 tree depth and balanced class weighting; and number 4 was a soft voting ensemble with weights of 1,2,2 for the respective Logistic Regression, Random Forest and Extra Trees models. For the hospital task, a sixth model - another Random Forest - was added with a balanced subsample class weight and min leaf sample size set to 2. One the hospital task we also used a Random Forest with 'balanced subsample' weighting - but then we also tried a different min leaf sample size.

To put soft-voting into practice we use a weighted average of the component probabilities - the class with the highest ensemble probability was the one we chose. That weighted average looks like this: P_(ensemble) (y = k | x) = (P_lr(k|x) + 2 * P_rf(k|x) + 2 * P_et(k|x)) / 5.

The data is split, 80% for training 20% for testing, with a random number generator seed of 42. This is done so that the two parts of the data have the same class distributions and we end up with a more realistic benchmark for the minority class. Both the data pre-processing steps are handled by scikit-learn

so that only the training data is used to fit the imputer, scaler and encoder. If we do the pre-processing before splitting the data, we run the risk of introducing the test results into our model and getting overly optimistic scores.

In terms of numeric predictors, we fill in missing values with the median - this is worked out from the training set. StandardScaler then takes a numeric value and turns it into a z score - this is the value subtracted by the mean and then divided by the standard deviation, based on the training set. For categorical predictors, missing values are replaced with the most common category that turned up in the training data. Our OneHotEncoder is set up to ignore any unknown categories that might show up when the model is put into practice and results in the creation of binary indicator columns. If there are any categories that show up in the test data but not in the training data, this prevents any errors popping up at inference time. We stick to the same pipeline design for all of our model comparisons.

Logistic Regression gives us a pretty simple linear baseline to work with - it estimates the probability of a particular class based on a linear combination of the features and so is a good benchmark because it's decision structure is smaller than a tree based model. It's been implemented with a maximum number of iterations of 1000 and class weights set to be equal. Random Forest then builds a bunch of decision trees and uses their predictions to come up with a final result - it uses 100 trees, is seeded with 42, has class weighting, and uses all available cores. Extra Trees does a similar thing to Random Forest but introduces a bit more randomness into the process of selecting splits. Both tree based methods are able to pick up complex relationships and interactions between features that might get missed by a linear classifier.

IV. EVALUATION METRICS

Accuracy is the overall number of correct predictions out of the total number of predictions made. Precision is how often you actually get a true positive result when you predict one. Your recall, or sensitivity, is how often you actually do catch all the real positives. F1 is the middle ground between precision and recall, giving you some idea of how well your model is doing on both fronts. Specificity is a good indicator of whether your model is good at keeping track of what it knows isn't true, keeping a handle on the negatives just as tightly.

F1 is the harmonic mean of precision and recall which means its a number that combines the two to give a better idea of a model's performance. Macro-averaged metrics for precision, recall and F1 give each class (in the case of the multiclass diabetes comparison) the same weight and are used kind of like a broad brush to get an idea of how well a model is doing. You can find all of these metrics in the notebook's standard model table for both datasets. The separate table for the tuned readmission model gives just the positive class metrics because for the readmission task, it turns out that most of the time when someone's not supposed to be readmitted (less than 30 days) they're not. That's only about 11.2% of the time. So a high accuracy score might look pretty good but they could be getting by on missing lots of early readmissions.

Accuracy is calculated as (the number of true positives plus true negatives) divided by the total (true positives plus true negatives plus false positives plus false negatives). Precision is the number of true positives divided by (true positives plus false positives). And your recall, or sensitivity, is the number of true positives divided by (true positives plus false negatives). F1 is a special number that combines the two - its calculated as 2PR divided by (P+R) which is a measure of both precision and recall. Specificity is the number of true negatives divided by (true negatives plus false positives) which gives you a sense of how well your model is keeping an eye on things that arent supposed to be true. ROC-AUC ,on the other hand, is a measure of the models ability to tell apart the true positives from the true negatives - its a kind of ranking system that shows how well the model can spot the real positives when they come up. With so many different metrics out there no single one is ever going to be enough to tell the whole story - especially when you have false negatives and false positives that have different consequences in the real world.

ROC-AUC and the confusion matrix are used as threshold based measures. As seen in the original readmission ensemble, the model could have moderate ranking discrimination but a poor default threshold. On the other hand, setting a lower threshold for what counts as a positive encounter might turn up more instances of people having a good time , but it's likely to pull in a lot of false alarms too. In a real-world service, figuring out just where to set that threshold would depend on a bunch of factors - how well interventions actually work, how much time and effort goes into trying to reach people, and what the cost is of ignoring the early warning signs of readmissions.

V. RESULTS

A. Dataset 1—Diabetes Status

Table II gives us a rundown on how the four classifiers did on a held-out test we set aside. And it was the Random Forest algorithm that came out on top overall, with a pretty impressive accuracy of 82.31%. But our proposed ensemble model wasn't far behind, with an accuracy of 82.197%. Whats a bit odd is that the Logistic Regression model actually had better results when it came to Macro F1 (0.4257) and ROC-AUC (0.7607). Meanwhile our proposed model lagged behind on the Macro F1 front with a rather disappointing 0.4030 - and it's ROC-AUC of 0.7398 didn't do much to make up for it.So, while the proposed ensemble model had some strengths, it wasn't necessarily the best performer when it came to the more class-balanced measures.

Table II. Dataset 1 test-set model comparison (macro-averaged precision, recall and F1).

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Logistic Regression	0.6292	0.4475	0.5135	0.4257	0.7607
Random Forest	0.8231	0.4376	0.3734	0.3771	0.7142
Extra Trees	0.8112	0.4189	0.3757	0.3805	0.6920
Proposed Ensemble	0.8197	0.4405	0.3945	0.4030	0.7398

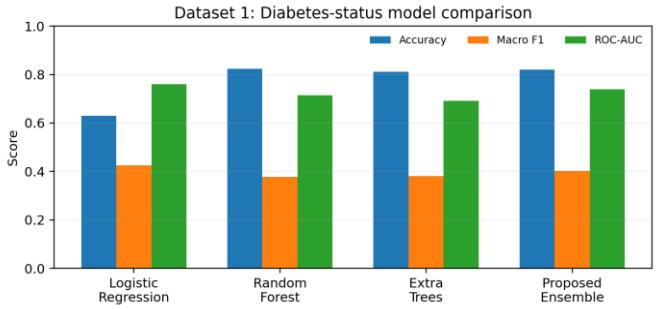


Fig. 2. Diabetes-status accuracy, macro F1 and ROC-AUC by classifier.

Similarly, the class level F1 for the proposed ensemble is 0.90 for no diabetes, 0.00 for prediabetes, and 0.31 for diabetes. There were 38,012 cases of no diabetes, 926 cases of prediabetes, and 7,019 cases of diabetes in the test set. The high overall accuracy is for the majority class and it is quite difficult to detect prediabetes in the models. The three-fold cross-validation macro F1 is higher for Logistic Regression (mean 0.4211, standard deviation 0.0019) over the weighted ensemble (mean 0.4017, standard deviation 0.0004).

B. Dataset 2—30-Day Readmission, Default Threshold

Table II summarizes the held out tests for the four classifiers. Random Forest yielded the highest accuracy (0,8231), followed by the proposed ensemble (0,82197). But, Logistic Regression had the highest macro F1 (0,4257) and ROC-AUC (0,7607). The ensemble's macro F1 was 0.4030, whereas its ROC-AUC was 0.7398, so it was not superior to Logistic Regression in the more class-balanced measures.

Table III. Dataset 2 default-threshold model comparison. Macro metrics summarize both binary classes; specificity was calculated by the notebook for the original results.

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Logistic Regression	0.6418	0.5430	0.6011	0.5095	0.6440
Random Forest	0.8887	0.7602	0.5024	0.4757	0.6511
Extra Trees	0.8888	0.7255	0.5064	0.4843	0.6345
Proposed Ensemble	0.8892	0.7545	0.5079	0.4873	0.6653

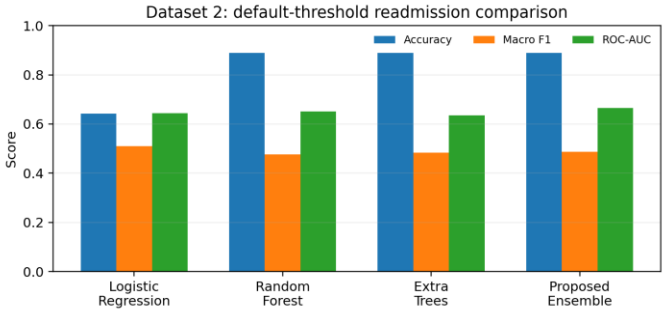


Fig. 3. Default-threshold readmission model comparison. High accuracy is not equivalent to strong positive-class detection.

Table IV. Positive-class metrics from the notebook's comparison of the original ensemble and class-weighted Random Forest.

Default-threshold model	Accuracy	Early-readmission precision	Early-readmission recall	Early-readmission F1	ROC-AUC
Original Ensemble	0.8892	0.6190	0.0172	0.0334	0.6653
Balanced Random Forest	0.8500	0.2689	0.2004	0.2296	0.6710

C. Dataset 2—Threshold-Tuned Balanced Random Forest

The search for the validation threshold was 0.40; its precision was 0.1795, recall was 0.5096 and F1 was 0.2655. On the untuned set of test data, the model with the threshold was 0.6876 with precision 0.1856, recall 0.5310, F1 0.2750 and ROC-AUC 0.6671. As with the original group, recall was increased substantially and precision and accuracy decreased. It is in fact a sensitivity-false-alarm trade-off rather than an overall increase.

Table V. Final held-out test results for the threshold-tuned Balanced Random Forest. All numbers are from the notebook's last cell of evaluation and from the file final_readmission_test_metrics.csv.

Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
0.40	0.6876	0.1856	0.5310	0.2750	0.6671

When it comes to early hospital readmission, the original ensemble actually came out with a high accuracy score - but at the cost of basically not detecting any cases at the default setting. Tweaking the threshold on the balanced random forest search actually brought in a lot more positive cases - 53.10% to be exact - but that was at the cost of a pretty dismal 18.56% precision rate and a pretty low F1 score of 0.2750, with a whole lot of unnecessary false alarm noise. The upshot of this was that we came to the conclusion that when it comes to healthcare model evaluation, you really do need to keep a close eye on how it's going to be used and what the real-life consequences might be. So, in the future - and I really hope its sooner rather than later - we need to start beefing up the patient-level validation, using some external validation sets, and take a closer look at the calibration

REFERENCES

[1] “Predicting 30-Day Hospital Readmission in Patients With Diabetes Using Machine Learning on Electronic Health Record Data,” Cureus, vol. 17, no. 4, e82437, 2025, doi: 10.7759/cureus.82437.

[2] “An interpretable machine learning approach for predicting 30-day readmission after stroke,” International Journal of Medical Informatics, vol. 174, art. 105050, 2023, doi: 10.1016/j.ijmedinf.2023.105050.

[3] “Machine learning for predicting readmission risk among the frail: Explainable AI for healthcare,” Patterns, vol. 3, no. 1, art. 100395, 2022, doi: 10.1016/j.patter.2021.100395.

[4] “An interpretable imbalance ensemble classification method for readmission risk assessment incorporating multi-view perturbation and SHAP analysis,” Decision Support Systems, vol. 190, art. 114404, 2025, doi: 10.1016/j.dss.2025.114404.

[5] G. Du et al., “Towards graph-based class-imbalance learning for hospital readmission,” Expert Systems with Applications, 2021, doi: 10.1016/j.eswa.2021.114791.

[6] “A Machine Learning Approach for Predicting 30-Day Hospital Readmission in Patients with Diabetes,” Healthcare, vol. 14, no. 9, art. 1185, 2026, doi: 10.3390/healthcare14091185.

[7] “An Innovative Ensemble Deep Learning Clinical Decision Support System for Diabetes Prediction,” IEEE Access, vol. 12, pp. 106193–106210, 2024, doi: 10.1109/ACCESS.2024.3436641.

[8] “A novel machine learning approach for diagnosing diabetes with a self-explainable interface,” Healthcare Analytics, vol. 5, art. 100301, 2024, doi: 10.1016/j.health.2024.100301.

[9] M. Kutlu, T. B. Donmez, and C. Freeman, “Machine Learning Interpretability in Diabetes Risk Assessment: A SHAP Analysis,” Computers and Electronics in Medicine, vol. 1, no. 1, pp. 34–44, 2024, doi: 10.69882/adba.cem.2024075.

[10] “Explainable AI Applied in Healthcare: A Case Study of Diabetes Prediction,” in Proc. IEEE QRS-C, 2024, pp. 336–344, doi: 10.1109/QRS-C63300.2024.00050.

[11] “Integrating Machine Learning with Explainable AI in Healthcare Analytics for Diabetes Prediction,” in Proc. ICEDEG, 2025, doi: 10.1109/ICEDEG65568.2025.11081581.

[12] P. Shah et al., “Predicting cardiovascular risk with hybrid ensemble learning and explainable AI,” Scientific Reports, vol. 15, art. 17927, 2025, doi: 10.1038/s41598-025-01650-7.

[13] “Explainable ensemble learning model for cardiovascular disease prediction with feature optimization and data balancing,” Discover Computing, vol. 29, art. 44, 2026, doi: 10.1007/s10791-026-09930-0.

[14] “A Robust and Explainable Framework for Predicting Hospital Readmission in Patients with Diabetes,” 2026. Available: https://www.mdpi.com/2673-4540/7/10/197.

[15] “Predictive Analytics for Diabetic Patient Care: Leveraging AI to Forecast Readmission and Hospital Stays,” 2025. Available through ScienceDirect.